SUPPORT D'EXERCICE

Les activités de l'atelier

Les consignes et les prompts de départ des quatre activités. Faites-les avec l'agent de code ou l'assistant que vous utilisez d'habitude, sur votre propre dépôt quand c'est demandé.

Activité 1 — Regenerate (10 min)

But : sentir que la sortie est une distribution, et que quelques mots la déplacent.

1. Écrivez une demande de code courte, volontairement **floue**.
2. Relancez-la **5 fois**. Notez ce qui reste stable.
3. Changez **3 mots** au maximum : langage, contrainte, format.
4. Ajoutez un **intrus** (`Fortnite` , `Victor Hugo` , un bout de poésie) et comparez.

```
Écris une fonction qui valide un email.
```

puis :

```
En TypeScript, sans regex, avec tests :
écris une fonction qui valide un email.
```

Autres points de départ : « Écris un script qui renomme des fichiers » (+ « en Bash, idempotent, avec --dry-run ») ; « Propose un schéma de table pour des réservations » (+ « PostgreSQL, avec index »).

> Cherchez les mots qui **resserrent** la sortie et ceux qui la **contaminent**.

Activité 2 — Formaliser (25 min)

But : passer par une structure intermédiaire validée avant de générer un artefact, puis consolider le prompt.

1. Choisissez un domaine : métier ou ludique (inventaire, backlog, plantes, jeux).
2. Faites produire une **structure intermédiaire** (YAML, types, JSON Schema, OpenAPI), sans code.
3. **Corrigez-la**, puis faites générer l'artefact : page HTML single-file, CLI, mini-doc.
4. Écrivez le **prompt consolidé** et testez-le sur un second domaine, dans une **session vierge**.

```
Propose une structure YAML minimale pour <domaine>.
Ne génère pas encore de code.
```

puis, une fois la structure corrigée :

```
Structure validée. Génère une page HTML single-file,
sans dépendance, avec un filtre sur chaque champ.
```

Critère de réussite	Vérifié ?
Le prompt final contient la structure intermédiaire, les règles de sortie et un exemple du format attendu	
Il marche dans une session vierge, sans l'historique de mise au point	
Il donne une qualité comparable sur deux domaines	

Activité 3 — Cartographier (25 min)

But : préparer une context window dense pour une tâche réelle, et mesurer l'effet.

1. **Cibler** une tâche réelle de votre dépôt : ajouter un endpoint, réduire une duplication, écrire des tests.
2. **Générer** une carte orientée tâche.
3. **Relire et corriger** : vérifier chaque nom, retirer le bruit.
4. **Tester** : nouvelle session, carte chargée, même tâche.

```
Je dois <tâche>. Avant de coder, écris une carte de cette tâche
en moins de 40 lignes : fichiers clés, commandes pour vérifier,
conventions à suivre, invariants, pièges connus.
Ne modifie aucun fichier.
```

Critère de réussite	Vérifié ?
La carte tient sur un écran et chaque chemin cité existe	
Au moins une correction humaine visible	
Dans la nouvelle session, l'agent lit moins de fichiers avant de proposer un plan	

Activité 4 — Extraire (20 min)

But : transformer une session qui a galéré en contexte durable et relu.

1. **Identifier** les frottements : tool mal appelé, API mal comprise, doc manquante, erreur répétée.
2. **Extraire** un seul output ciblé.
3. **Relire** : élaguer, corriger, densifier.

```
Extrais de cette session un troubleshooting :
pour chaque erreur rencontrée, une ligne « erreur → cause → fix ».
Garde seulement ce qui a été vérifié.
Si une cause est supposée, écris-le. N'invente rien.
```

Fichier	Contenu
`user-guide.md`	comment utiliser l'outil, commandes qui marchent, pièges
`troubleshooting.md`	erreur → cause → fix
`api-improvements.md`	signatures, noms, messages d'erreur, docs à corriger
`new-tool.md`	le script ou le tool qui aurait évité le frottement
`next-session.md`	ce qu'on recharge dans une session propre

> Une extraction **non relue** n'est jamais une source d'autorité.

Où ranger ce que vous avez produit

Contexte	Contenu	Chargé
Instructions projet (`AGENTS.md`)	conventions, commandes, pièges du dépôt	à chaque session
Carte de tâche	fichiers clés, invariants, plan d'une tâche	pour cette tâche
Skill	prompt consolidé, exemples, scripts	à la demande

© Zenika · Context Engineering — reprendre le contrôle des agents de code